

05

BOOK 05 / 08

입력과 출력의 안전한 처리

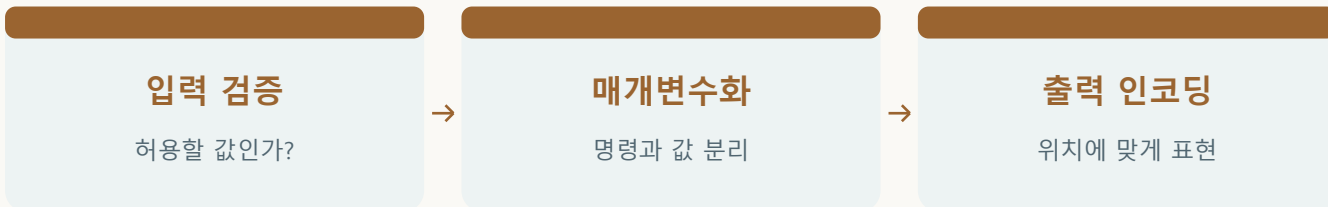
입력의 의미와 사용 위치를 구분하기

개념 이해 / 사례 읽기 / 확인 문제 약 15분

비공개 학습 노트 서비스를 따라 배우는 보안의 기본

이 책을 읽는 방법

검증·매개변수화·인코딩은 역할이 다릅니다.



읽기 순서

학습 목표와 선수 지식	3
시작 상황: 작은따옴표를 모두 지우면 해결될까?	3
개념 설명	3
예제 1: 허용할 제목을 명확하게 정의하기	4
예제 2: SQL 구조와 값을 분리하기	4
예제 3: HTML 본문에서 제목을 문자로 표시하기	6
개발·검수 시 확인사항	6
핵심 요약	6
확인 문제	7
정답과 해설	8
다음 학습과 출처	8

사례를 먼저 읽고, 개념과 코드의 차이를 확인한 뒤 문제를 풀어 보세요. 정답과 해설은 문제 다음 페이지에서 시작합니다.

공통 사례 / 비공개 학습 노트

민서(사용자 101)의 노트는 10번, 준호(202)의 노트는 20번입니다. 모든 노트는 비공개이며, 로그인한 소유자만 조회·수정할 수 있습니다. 제목은 1~80자의 일반 텍스트입니다.

비밀번호·세션 토큰은 로그에 남기지 않고, 권한을 확인할 수 없으면 본문을 보내지 않습니다. 이 가상 서비스의 예제는 학습용 축약본이며 실제 서비스의 계정·데이터·구현과 무관합니다.

학습 목표와 선수 지식

문자열과 함수 호출을 읽을 수 있다고 가정합니다. SQL은 DB에 조회·변경을 요청하는 언어이며, HTML은 브라우저가 화면 구조를 해석하는 형식이라는 점을 알아 두면 됩니다. 학습 후에는 입력 검증·매개변수화·출력 인코딩의 목적을 구분할 수 있습니다.

시작 상황: 작은따옴표를 모두 지우면 해결될까?

노트 제목에 작은따옴표가 들어가면 DB 조회가 실패하자 개발자가 모든 작은따옴표를 지우기로 했습니다. 그러면 **오늘의 '핵심' 정리** 같은 정상 제목도 바뀝니다. 어떤 문자가 위험한지는 그 값을 해석하는 위치에 따라 달라집니다. 무조건 문자를 지우기 전에 값이 어떻게 사용되는지 확인해야 합니다.

개념 설명

방법	목적	노트 서비스의 예
입력 검증	서비스가 허용하는 값의 종류·길이·범위·의미인지 확인	제목은 문자열이고 1~80자이며 공백만으로 구성되지 않음
매개변수화	명령 구조와 데이터 값을 분리해서 전달	SQL 질의에 제목을 문자열로 이어 붙이지 않고 값 인자로 전달
출력 인코딩	출력 위치의 문법으로 해석될 문자를 데이터로 표현	HTML 본문에 제목의 <를 문자로 표시

제목 길이가 올바르다고 해서 SQL이나 HTML에서 안전하게 처리된다는 뜻은 아닙니다. 반대로 SQL 매개변수화를 했다고 해서 100만 자의 제목을 받아도 된다는 뜻은 아닙니다. 세 방법은 서로 다른 문제를 다룹니다.

HTML 문법을 일부 허용하는 편집기는 별도의 HTML 정화 정책이 필요합니다. 이 과정의 제목은 일반 텍스트이므로 그런 기능을 가정하지 않습니다.

예제 1: 허용할 제목을 명확하게 정의하기

PYTHON / 실행 예제

```
def valid_title(value):
    return (
        isinstance(value, str)
        and 1 <= len(value) <= 80
        and bool(value.strip())
    )

print(valid_title("오늘의 '핵심' 정리")) # True
print(valid_title(" "))                # False
print(valid_title("가" * 81))           # False
print(valid_title(None))                # False
```

이 검사는 제목의 기본 정책만 확인합니다. Python의 `len`은 여기서 Unicode 코드 포인트 수를 셉니다. 사용자에게 보이는 글자 묶음 수나 UTF-8 바이트 수와 항상 같지는 않습니다. 실제 서비스에서는 화면·서버·DB 사이의 길이 기준도 맞춰야 합니다.

예제 2: SQL 구조와 값을 분리하기

아래 예제는 메모리 안에서만 동작하고 파일이나 외부 DB에 접근하지 않습니다. `current_user_id`는 인증된 사용자라고 가정하며, 두 방식 모두 소유자 조건을 포함해 제목 처리 방식의 차이에 집중합니다.

PYTHON / 실행 예제

```
import sqlite3

db = sqlite3.connect(":memory:")
try:
    db.execute("CREATE TABLE notes (owner_id INTEGER, title TEXT)")
    db.execute(
        "INSERT INTO notes VALUES (?, ?)",
        (101, "오늘의 '핵심' 정리"),
    )
    current_user_id = 101
    title = "오늘의 '핵심' 정리"

    # 취약한 방식: 제목을 SQL 문법 안에 직접 넣음
    bad_sql = (
        "SELECT title FROM notes WHERE owner_id = ? "
        f"AND title = '{title}'"
    )
    try:
        db.execute(bad_sql, (current_user_id,)).fetchall()
    except sqlite3.OperationalError:
        print("문자열 조합 방식: SQL 해석 실패")

    # 개선한 방식: 제목도 SQL과 분리된 값으로 전달
    rows = db.execute(
        "SELECT title FROM notes WHERE owner_id = ? AND title = ?",
        (current_user_id, title),
    ).fetchall()
    print(rows) # [("오늘의 '핵심' 정리",)]
finally:
    db.close()
```

첫 방식에서는 정상 제목의 따옴표조차 SQL 구조에 영향을 줍니다. 두 번째 방식에서는 제목 전체가 값으로 전달됩니다. 악의적인 입력이 질의 구조를 바꾸는 원리는 SQL 삽입 심화 단원에서 다룹니다.

이 예제의 ?는 Python `sqlite3`의 값 자리표시자입니다. 다른 DB 드라이버는 표기법이 다를 수 있습니다. 테이블 불명이나 정렬 방향 같은 SQL 구조 요소까지 값 매개변수로 바꿀 수 있는 것은 아니므로, 동적 구조는 별도의 허용 목록 등으로 설계해야 합니다.

예제 3: HTML 본문에서 제목을 문자로 표시하기

PYTHON / 실행 예제

```
from html import escape

title = "<b>중요</b> & 복습"
unsafe_html = f"<h1>{title}</h1>"
safer_html = f"<h1>{escape(title)}</h1>"
print(unsafe_html) # <h1><b>중요</b> & 복습</h1>
print(safer_html)  # <h1>&lt;b>&gt;중요&lt;/b>& & 복습</h1>
```

첫 결과를 HTML로 해석하면 사용자가 쓴 태그가 화면 구조에 들어갑니다. 두 번째는 제목을 HTML 본문의 텍스트로 표현합니다. 이 예제의 태그는 현상을 보여 주기 위한 것이며 스크립트를 실행하지 않습니다.

이 처리를 JavaScript 코드·CSS·임의의 URL에 그대로 적용해서는 안 됩니다. 출력 위치마다 규칙이 다릅니다. 템플릿 엔진이 자동 인코딩을 제공한다면 해당 기능의 범위를 확인하고 사용하며, 인코딩을 중복 적용하거나 '안전한 HTML'로 임의 표시하지 않습니다.

개발·검수 시 확인사항

정상 제목에 따옴표나 한글이 있어도 기능을 유지해야 합니다. 길이 초과는 정책에 따라 거부하고, 허용한 텍스트는 DB와 화면에서 데이터로 다루는지 각각 확인합니다. 검증 오류를 이유로 입력을 조용히 변경하면 사용자가 작성한 의미가 달라질 수 있습니다.

핵심 요약

- 입력 검증은 값의 허용 조건을 확인합니다.
- SQL 매개변수화는 명령과 값을 분리합니다.
- 출력 인코딩은 출력 위치에 맞춰 적용합니다. 세 방법은 서로를 대체하지 않습니다.

확인 문제

1. 제목 길이를 80자로 제한했으므로 SQL에 직접 이어 붙여도 된다는 주장은 맞을까요?

2. SQL 매개변수로 저장한 제목을 화면에 보여 줄 때 HTML 처리가 필요할까요?

3. 작은따옴표가 있는 정상 제목을 모두 거부하는 대신 무엇을 확인해야 할까요?

정답과 해설

1. 아닙니다. 길이 검사는 SQL 구조와 값의 분리를 보장하지 않습니다.
2. 필요합니다. DB 처리와 HTML 출력은 다른 해석 단계입니다.
3. 제목의 업무상 허용 조건과, DB에서 값을 매개변수로 전달하는지를 확인합니다. 따옴표 자체를 금지할 업무상 이유가 없다면 정상 입력을 보존해야 합니다.

다음 학습과 출처

입력 검증의 역할: [OWASP Input Validation](#). 값 바인딩: [Python sqlite3 공식 문서](#). HTML 문자 변환: [Python html 공식 문서](#). 다음에는 처리한 정보가 어디에 남는지 살펴봅니다.

나의 한 문장

이 강의에서 배운 내용을 동료에게 설명한다면 어떻게 말할까요?